

Sequence Processing

Hrachya Asatryan

Sequence Processing

The Task: Sequence Classification (Sentiment Analysis)

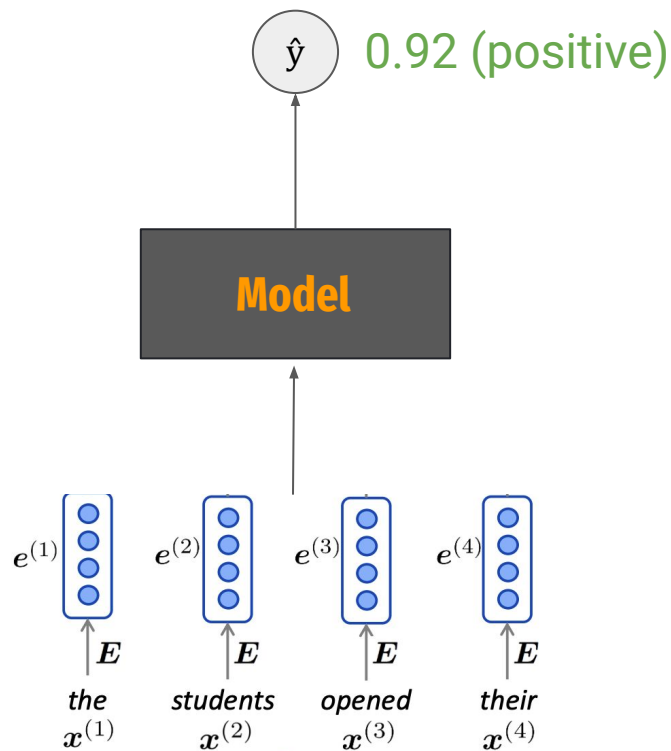
Before introducing a new architecture, we must define the mathematical objective.

We are performing a **Many-to-One sequence mapping**:

- **Input:** A variable-length sequence of T tokens, represented by their dense embedding vectors:
 $X = (x_1, x_2, \dots, x_T)$ where $x_t \in \mathbb{R}^d$
- **Target:** A ground-truth binary label $y \in \{0, 1\}$.
- **Output:** A model prediction representing the probability of a positive class: $\hat{y} = P(y=1 \mid x_1, \dots, x_T)$, where $\hat{y} \in [0, 1]$.

The core challenge: T is not fixed. A review can be 5 words or 5,000 words.

Sequence Processing



Sequence Processing

The Bag of Words Failure: Why not average the vectors? The simplest approach to handle a variable-length sequence X is to average the embeddings into a single fixed-size vector, then pass it to a standard Feed-Forward Neural Network.

$$\bar{x} = \frac{1}{T} \sum_{t=1}^T x_t$$

The fatal flaw: Vector addition is commutative. This destroys all temporal and syntactic order.

- **Sentence A:** "The movie was good, not bad." $\rightarrow y = 1$
- **Sentence B:** "The movie was bad, not good." $\rightarrow y = 0$

For both A and B, $\bar{x}_A = \bar{x}_B$. The network receives the exact same input for completely opposite targets.

Sequence Processing

What about Convolutional Neural Networks (CNNs)?

Since we just covered CNNs, can spatial weight-sharing solve this?

Yes and no. **1D Convolutions** act as excellent n-gram feature extractors. A kernel of size $k=2$ can easily learn to detect the bi-gram "not good."

The Limitation: Fixed Receptive Fields.

A CNN applies filters locally. But language has unbounded, long-range dependencies.

- *Example:* "The movie, which I initially thought was going to be a complete disaster due to the terrible trailer, was actually **good**."

To connect "movie" to "good" over a 20-word gap, a CNN would need massive, overlapping kernels or extremely deep dilated architectures. We need an architecture natively built for *time*.

Sequence Processing

this →	0.2	0.4	-0.3
movie →	0.1	0.2	0.6
has →	-0.1	0.4	-0.1
amazing →	0.7	-0.5	0.4
diverse →	0.1	-0.2	0.1
characters →	0.6	-0.3	0.8

Recurrent Neural Networks

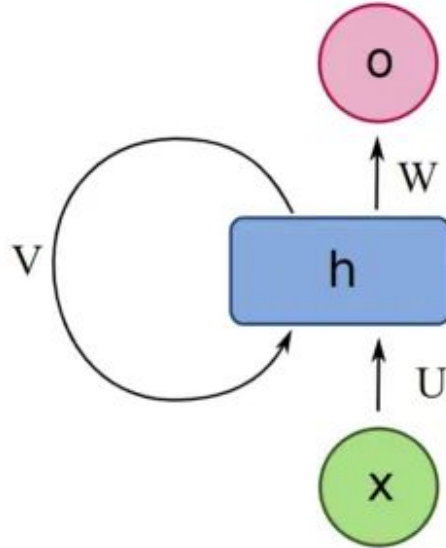
The Core Concept: Temporal Weight Sharing

If 1D CNNs give us *spatial* weight sharing (sliding a filter across different positions), we need a mechanism for *temporal* weight sharing to handle variable-length time series.

Enter the **Recurrent Neural Network (RNN)**.

Instead of processing the entire sequence at once, an RNN processes the sequence **iteratively**, one token at a time. To ensure that token x_2 has context from token x_1 , the network maintains an internal state; a "working memory."

Recurrent Neural Networks



Recurrent Neural Networks

The Hidden State (h_t): The Working Memory

At any given time step t , the network computes its current hidden state h_t by looking at two things:

1. The current input token: x_t
2. The previous hidden state: h_{t-1}

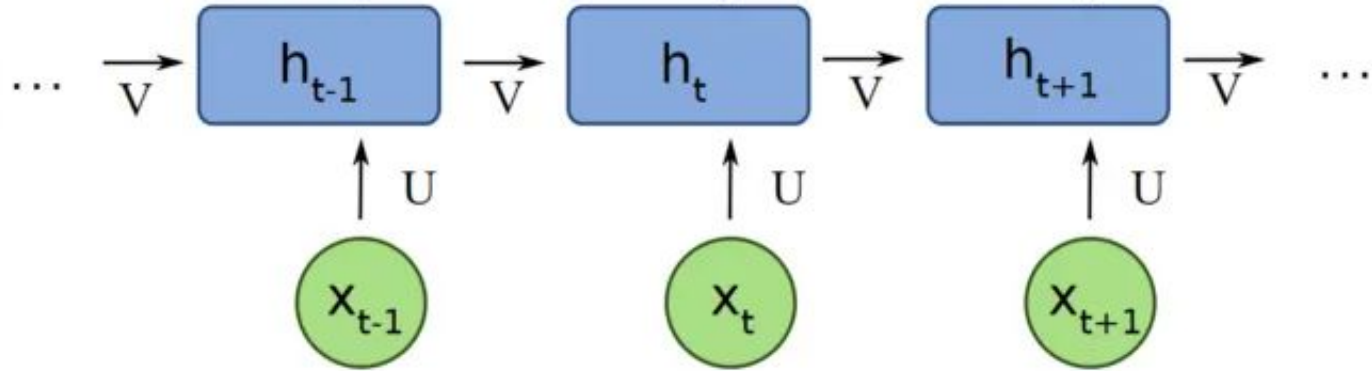
By definition, h_{t-1} contains the accumulated information from $x_1, x_2, \dots, x_{t-1}$. Therefore, h_t becomes a mathematical summary of the entire sequence *up to time t* .

We can view the RNN as a recursive function:

$$h_t = f_W(h_{t-1}, x_t)$$

Where W represents the parameters of our network.

Recurrent Neural Networks



Recurrent Neural Networks

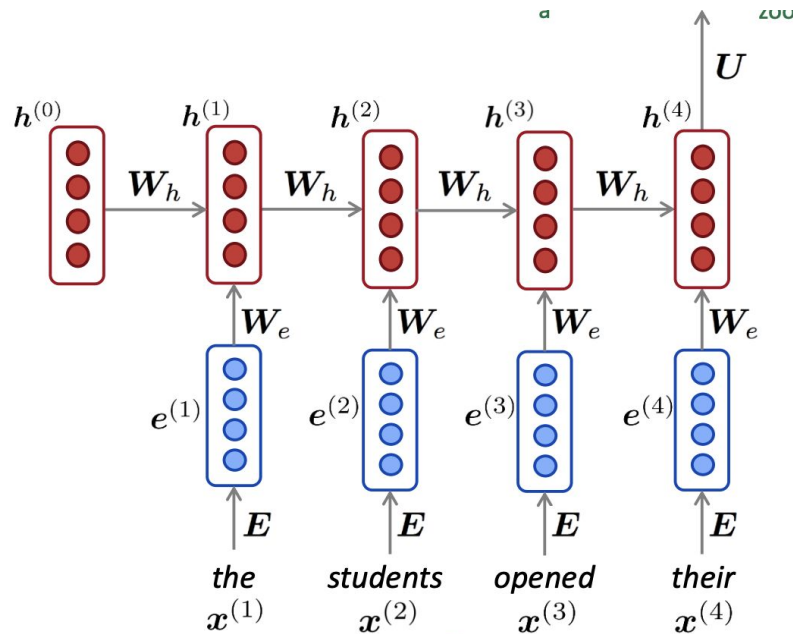
The Forward Pass Mathematics

Let's define the exact linear algebra inside that recursive function f_w . To compute the new hidden state, we apply a linear transformation followed by a non-linear activation function (typically $\tanh$ for RNNs to keep values bounded between $[-1, 1]$).

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- W_{hh} : The hidden-to-hidden weight matrix (how much we weigh the past).
- W_{xh} : The input-to-hidden weight matrix (how much we weigh the current word).
- b_h : The hidden bias vector.

Recurrent Neural Networks



Recurrent Neural Networks

Matrix Dimensions and Parameter Count

Let's assume our word embeddings are size d and we choose our hidden state to have size m .

- $x_t \in \mathbb{R}^d$
- $h_{t-1}, h_t \in \mathbb{R}^m$

This dictates our weight matrices:

- $W_{xh} \in \mathbb{R}^{m \times d}$
- $W_{hh} \in \mathbb{R}^{m \times m}$
- $b_h \in \mathbb{R}^m$

Crucial detail: The matrices W_{xh} and W_{hh} are exactly the same at $t=1$, $t=2$, and $t=100$. The parameters do not grow with the length of the sentence. This is what we mean by temporal parameter sharing.

Recurrent Neural Networks

Generating the Final Prediction (Many-to-One)

We have processed the entire sequence $X = (x_1, \dots, x_T)$. We now have a sequence of hidden states $(h_1, \dots, h_T)$.

For Sentiment Analysis, we do not care about intermediate predictions. We only care about the final summary of the review, which is captured in h_T .

We pass h_T through a standard dense layer to map our m -dimensional memory down to a scalar probability.

$$\hat{y} = \sigma(W_{hy}h_T + b_y)$$

- $W_{hy} \in \mathbb{R}^{1 \times m}$
- σ : The sigmoid activation function.

Recurrent Neural Networks

The Information Bottleneck

Consider what we have just built.

Regardless of whether the input sentence is 5 words long or 5,000 words long, the final classification decision $\hat{y}$ relies entirely on h_T .

h_T is a vector of fixed size (e.g., $m=256$).

This means an RNN is forced to compress the meaning, syntax, and sentiment of an arbitrarily long document into 256 floating-point numbers.

As we will see, this creates a severe bottleneck.

Problems with RNNs

The Short-Term Memory Failure We established that RNNs can theoretically process variable-length text. An RNN *should* be able to remember a subject from word 1 all the way to word 100.

In practice, standard RNNs suffer from severe amnesia. They heavily bias their final hidden state, h_T , toward the most recently processed words.

If the network reads a 50-word review that starts with "I absolutely despised this film..." but ends with "...however the cinematography was okay", a standard RNN will likely misclassify it as Positive.

Why? The answer lies in how we calculate the error during training: Backpropagation Through Time (BPTT).

Problems with RNNs

The Short-Term Memory Failure We established that RNNs can theoretically process variable-length text. An RNN *should* be able to remember a subject from word 1 all the way to word 100.

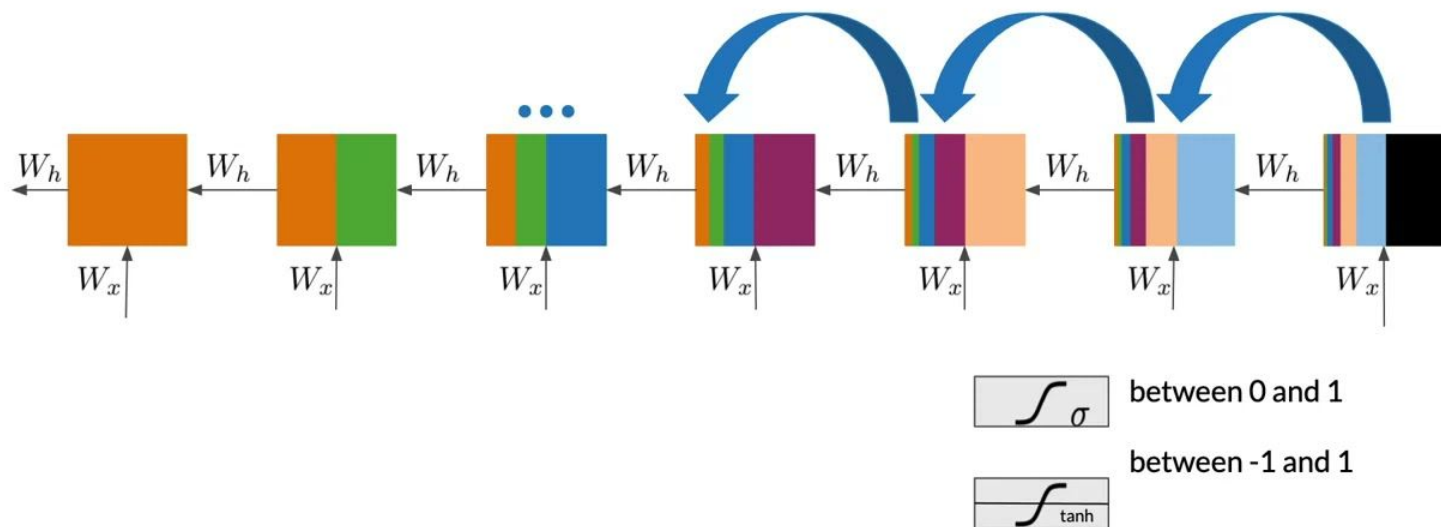
In practice, standard RNNs suffer from severe amnesia. They heavily bias their final hidden state, h_T , toward the most recently processed words.

If the network reads a 50-word review that starts with "I absolutely despised this film..." but ends with "...however the cinematography was okay", a standard RNN will likely misclassify it as Positive.

Why? The answer lies in how we calculate the error during training: Backpropagation Through Time (BPTT).

Problems with RNNs

Backpropagation through time



Problems with RNNs

Training the RNN: Backpropagation Through Time To train the network, we must update the weight matrices (W_{hh} , W_{xh}) by calculating the gradient of our loss function L with respect to those weights.

Because the network is unrolled over time, the loss at the final step T depends directly on h_T . But h_T depends on h_{T-1} , which depends on h_{T-2} , all the way back to h_1 .

By the chain rule of calculus, to find out how a change in the earliest hidden state (h_1) affects the final output, we must multiply the local gradients at every single intermediate time step.

Problems with RNNs

The Chain Rule Across Time Let's look at the gradient of the loss L with respect to the hidden-to-hidden weight matrix, W_{hh} . We must sum the contributions of the gradient at each time step t .

For a specific time step t , finding the gradient requires computing the derivative of h_t with respect to a past state h_k .

$$dh_t / dh_k = \prod_{i=k+1}^t dh_i / dh_{i-1}$$

This chain of multiplied derivatives is the mathematical root of the amnesia problem.

Problems with RNNs

Unpacking the Local Derivative What exactly is that local derivative, dh_i / dh_{i-1} ?

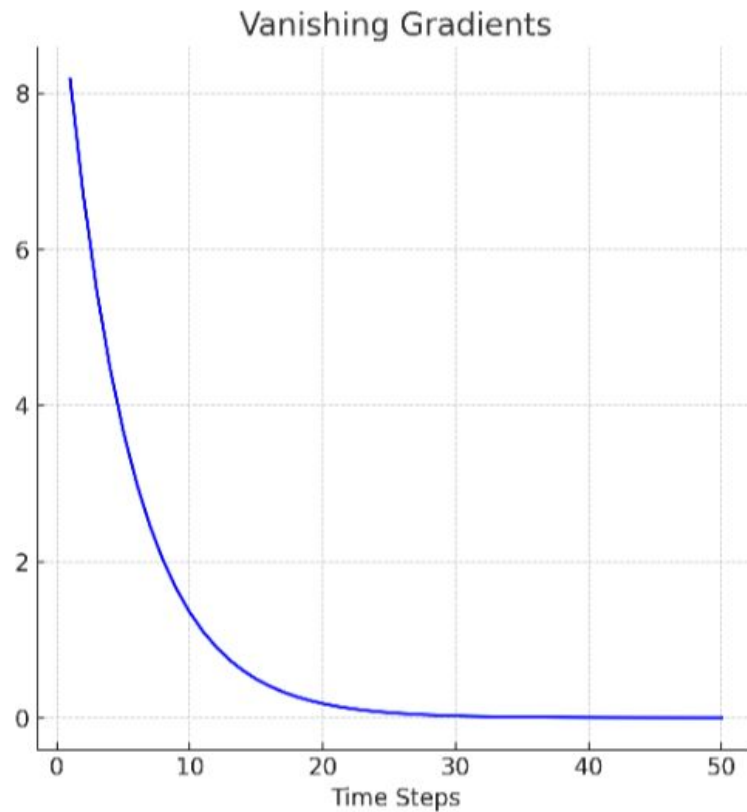
Recall our forward pass equation: $h_i = \tanh(W_{hh} * h_{i-1} + W_{xh} * x_i + b_h)$

Taking the derivative of h_i with respect to h_{i-1} requires applying the chain rule through the tanh activation function:

$$dh_i / dh_{i-1} = \text{diag}(1 - h_i^2) * W_{hh}$$

Here, $\text{diag}(1 - h_i^2)$ is simply the Jacobian matrix of the tanh activation function.

Problems with RNNs



Problems with RNNs

The Vanishing Gradient Problem Because the derivative of $\tanh$ is always strictly between 0 and 1, and the weight matrix eigenvalues are typically initialized to be less than 1, our gradient product collapses to zero very quickly.

When the gradient of the loss with respect to time step 1 goes to 0, it means the network receives NO error signal for those initial words.

If the network misclassifies the review because it forgot the word "despised" at step 1, the vanishing gradient means it physically cannot update the weights to fix that mistake. The bridge between the error and the input has collapsed.

Problems with RNNs

The Exploding Gradient (The Opposite Extreme) What if the largest eigenvalue of W_{hh} is initialized to be greater than 1?

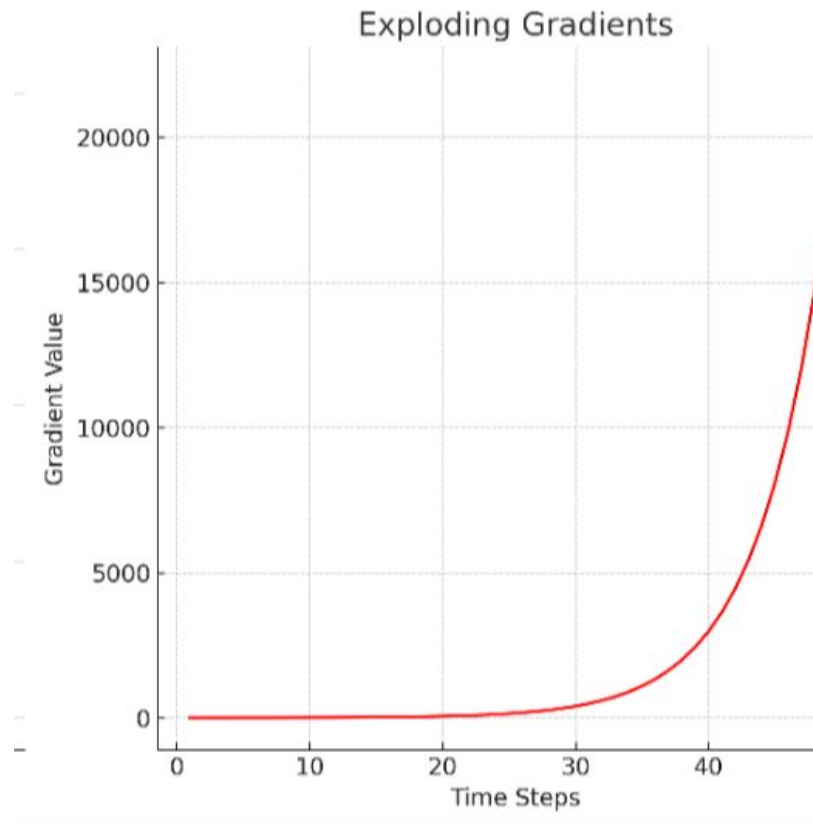
Then W_{hh}^T grows exponentially. This is the Exploding Gradient Problem. The gradients become so massive that weight updates overshoot wildly, completely destabilizing the network and causing the loss to output NaN (Not a Number).

We can patch exploding gradients with a crude trick called "Gradient Clipping" (simply capping the gradient vector's maximum value before updating).

But we cannot "clip" a vanished gradient back into existence. To solve amnesia, we need a fundamental architectural change.

This breakdown dives directly into the linear algebra and numerical stability issues that plague standard RNNs, setting up an incredibly strong motivation for why the LSTM cell state exists.

Problems with RNNs



Long Short-Term Memory (LSTM)

A New Architecture to Cure Amnesia We have established that multiplying a full weight matrix over and over causes gradients to vanish. To fix this, Sepp Hochreiter and Jürgen Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997.

The fundamental innovation of the LSTM is the separation of memory into two distinct channels:

1. The Hidden State (h_t): The short-term working memory (what we had in standard RNNs).
2. The Cell State (C_t): The long-term memory.

The Cell State is the key to solving the vanishing gradient problem.

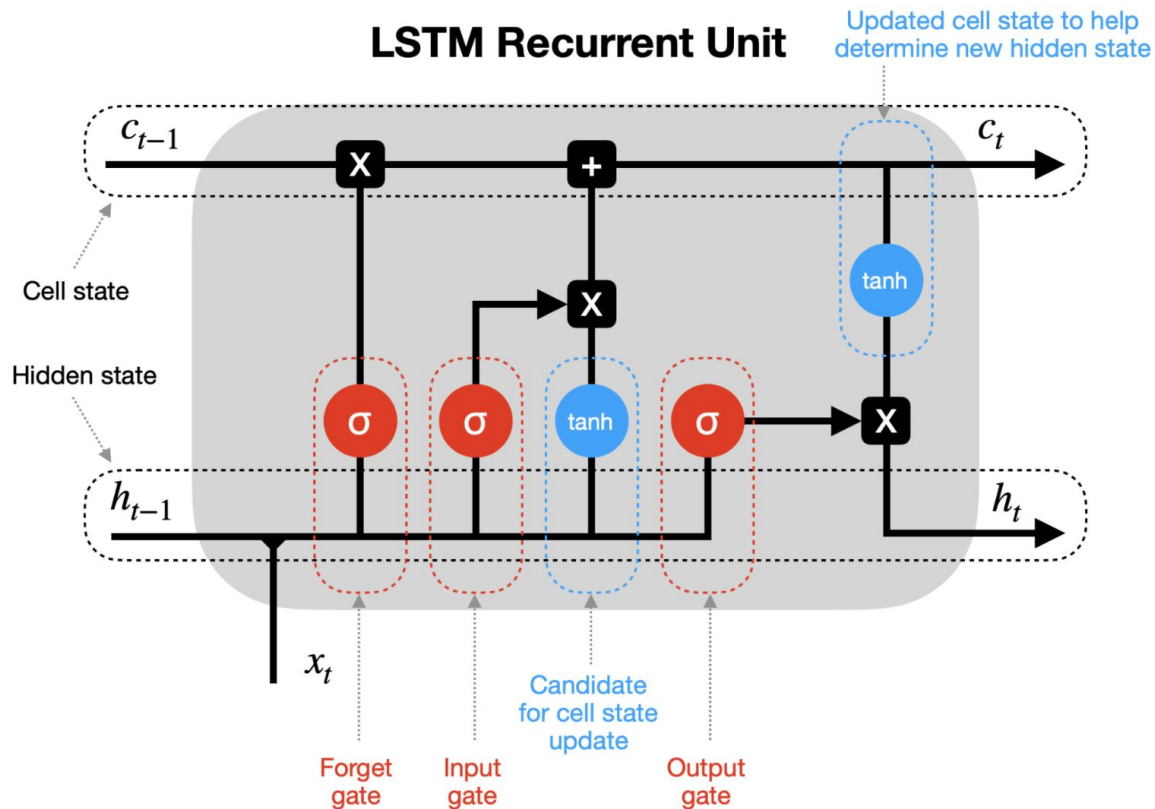
Long Short-Term Memory (LSTM)

The Cell State: The Conveyor Belt Think of the Cell State, C_t , as a conveyor belt that runs straight down the entire sequence of time steps.

Instead of passing the long-term memory through complex, destructive matrix multiplications and tanh activations at every step, the cell state only undergoes minor linear interactions—specifically, pointwise addition and multiplication.

Because information can flow down this conveyor belt unchanged, the error gradients during backpropagation can flow backwards across 100+ words without vanishing.

Long Short-Term Memory (LSTM)



Long Short-Term Memory (LSTM)

Controlling the Flow with Gates Of course, we cannot just let the conveyor belt run completely untouched. As we read a sentence, we need to selectively add new information to it, and remove old, irrelevant information from it.

The LSTM accomplishes this using "Gates".

A gate is simply a fully connected neural network layer with a sigmoid activation function. Because sigmoid squishes outputs between 0 and 1, a gate acts as a continuous bouncer:

- An output of 0 means "let nothing through".
- An output of 1 means "let everything through".

Long Short-Term Memory (LSTM)

Step 1: The Forget Gate

The very first thing an LSTM does at time step t is decide what information to wipe from the past long-term memory, C_{t-1} .

For example, if the network just read a period indicating the end of a sentence, the Forget Gate might output zeros to wipe the previous subject from memory.

It computes a forget vector, f_t , by looking at the previous short-term state and the current word:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

(Note: The brackets denote concatenating the two vectors into one longer vector before multiplying by the weight matrix W_f).

Long Short-Term Memory (LSTM)

Step 2: The Input Gate & Candidate Memory

Next, we decide what new information to add to the conveyor belt. This requires two parts:

1. The Input Gate (i_t): A sigmoid layer deciding *which* specific values we are going to update.
2. The Candidate Memory ($\tilde{C}_t$): A tanh layer creating a vector of *new candidate values* that could be added to the state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Long Short-Term Memory (LSTM)

Step 3: Updating the Cell State

Now we have all the pieces required to actually update the conveyor belt.

We take the old state C_{t-1} and pointwise multiply it by the forget gate f_t . Then, we pointwise multiply our new candidate values $\tilde{C}_t$ by the input gate i_t , and add the result to the belt.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Notice the operator $\odot$ (Hadamard product / pointwise multiplication). There is no full matrix multiplication here. This addition operation is exactly what saves the gradients from vanishing during BPTT.

Long Short-Term Memory (LSTM)

Step 4: The Output Gate (Generating the Hidden State)

We have successfully updated our long-term memory to C_t . Finally, we need to generate our short-term working memory, h_t , which is what we will pass to the next time step (and what we use for our final classification).

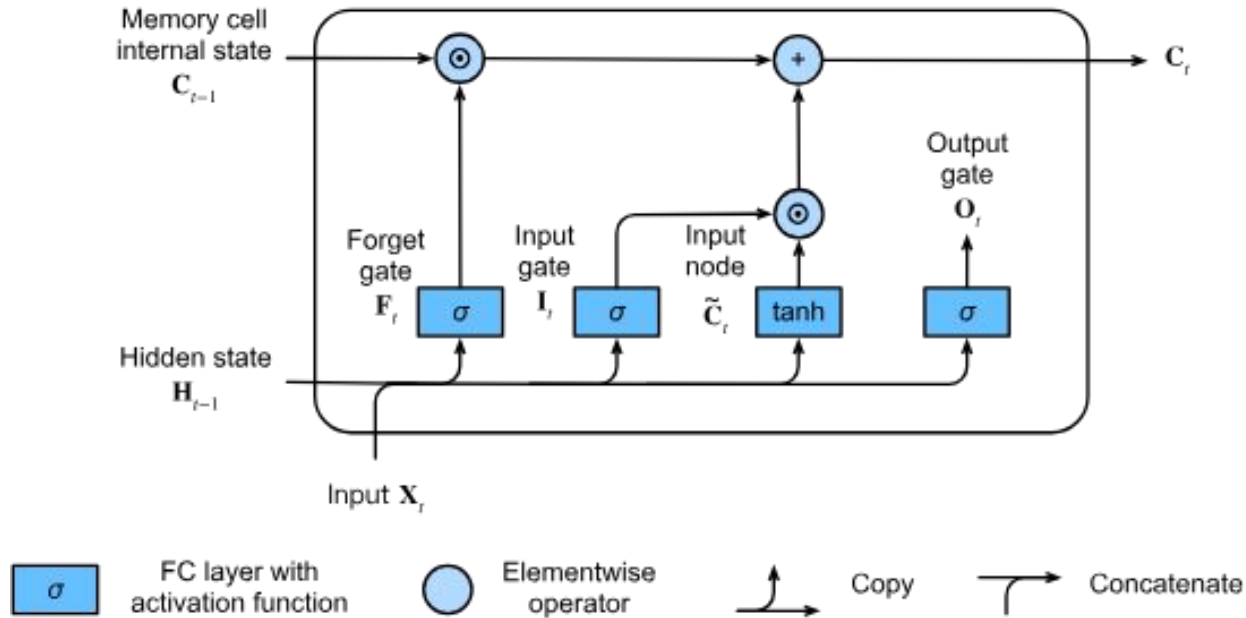
The hidden state is simply a filtered version of the cell state. We use an Output Gate (o_t) to decide which parts of the cell state to reveal.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

We push the cell state through a $\tanh$ to force the values back between -1 and 1, then multiply by the output gate.

Long Short-Term Memory (LSTM)



Long Short-Term Memory (LSTM)

The Preprocessing Paradigm Shift

Before we build the final classifier, we need to talk about the data we feed into it.

In early NLP (like Bag of Words or TF-IDF), we aggressively stripped away "stop words" (the, a, is, at) and lemmatized verbs (running $\rightarrow$ run). We did this to reduce the vocabulary size and save memory, because those models had no concept of sequence.

For Deep Sequential Models like LSTMs, **we stop doing this.**

Long Short-Term Memory (LSTM)

Why keep the "noise"? An LSTM relies on the exact sequence and structure of language to manipulate its gates.

If you delete stop words, you might delete the word "not". Without "not", the phrase "not good" becomes "good". The LSTM's Forget Gate was waiting for that "not" to wipe the positive sentiment, and you starved it of the trigger.

Furthermore, LSTMs have enough parameters to learn grammar, tense, and syntax. Lemmatizing words destroys the natural flow of the sentence. We want to feed the network raw, unedited, grammatically intact tokens.

Long Short-Term Memory (LSTM)

Building the Classifier: The Many-to-One Architecture We have an LSTM that can successfully remember information over long sequences without vanishing gradients. How do we hook this up to actually predict a sentiment score in PyTorch?

We use a Many-to-One architecture:

1. Pass the entire sequence of token embeddings, x_1 through x_T , into the LSTM.
2. The LSTM will output a hidden state for every single time step: $h_1, h_2, \dots, h_T$.
3. We **ignore** all intermediate hidden states (h_1 through h_{T-1}).
4. We extract only the final hidden state, h_T , because it contains the distilled long-term memory of the entire review.

Long Short-Term Memory (LSTM)

The Final Dense Layer

Once we have our final summary vector, h_T , we treat it exactly like the output of a Convolutional Neural Network before classification.

We pass h_T into a standard Fully Connected (Linear) layer, followed by a Sigmoid activation function to squash the output into a probability between 0.0 and 1.0.

$$\hat{y} = \sigma(W_{hy} \cdot h_T + b_y)$$

If $\hat{y} > 0.5$, we classify the review as Positive. Otherwise, Negative.

Long Short-Term Memory (LSTM)

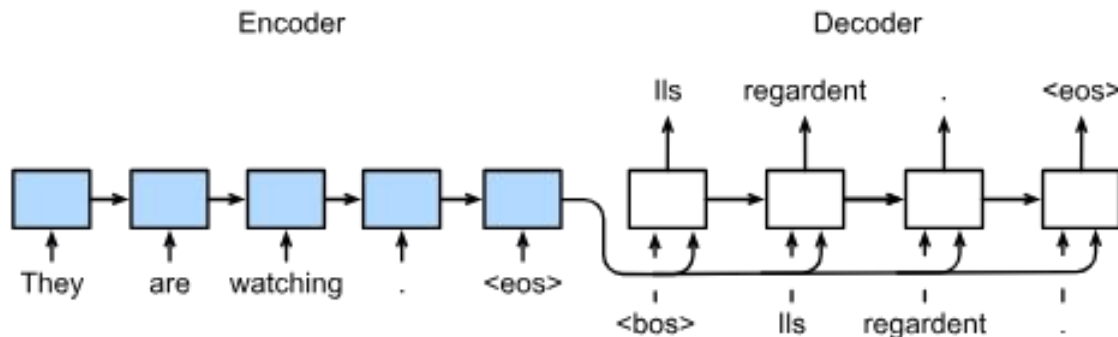
What about Batching? (Padding) One critical engineering detail: PyTorch and GPUs require operations to be done in fixed-size tensors (matrices).

If we have a batch of 32 movie reviews, they will all be different lengths. To push them through an LSTM at the same time, we must find the longest review in the batch, and add "Padding Tokens" (usually 0s) to the end of all the shorter reviews so they match the maximum length.

The LSTM processes the padding tokens, but we tell PyTorch to ignore the network's output on those empty steps.

Seq2Seq

- So far, we looked at a Many-to-One Problem.
- **Today's Problem:** How do we do Translation or Summarization? The input is a sequence (English), and the output is a completely different sequence of a different length (Armenian).
- **The Solution:** The **Sequence-to-Sequence (Seq2Seq)** architecture, introduced by Google in 2014. It uses two separate LSTMs: an **Encoder** and a **Decoder**.



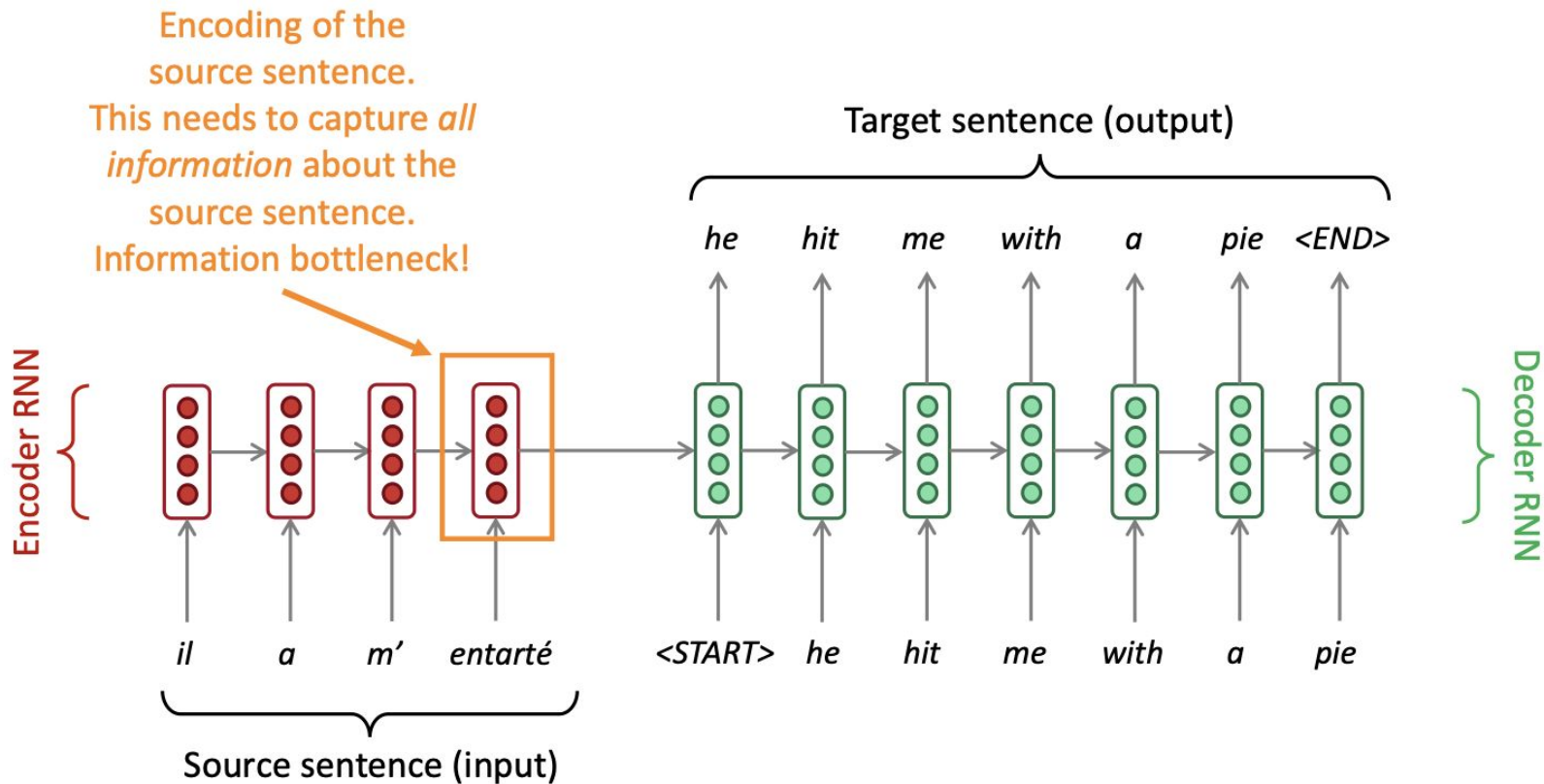
Seq2Seq

- **The Encoder:** Reads the input sequence $x_1, x_2, \dots, x_T$ step-by-step.
- It takes the *very last* hidden state and designates it as the **Context Vector (c)**.
- **The Decoder:** An entirely separate LSTM. Its initial hidden state is set to c. It generates the output sequence $y_1, y_2, \dots, y_T$ one word at a time, stopping when it outputs an **<EOS>** (End of Sequence) token.

The Information Bottleneck

- Look closely at the Context Vector c . It is a fixed-size mathematical vector (e.g., 256 or 512 dimensions).
- **The Problem:** We are forcing the Encoder to compress the meaning, syntax, and nuance of an *entire paragraph* into a single 512-dimensional array.
- **The Result:** The model suffers from severe amnesia on long sentences. The BLEU score (translation accuracy) plummets for sentences longer than 20 words. You simply cannot compress that much data without loss.

The Information Bottleneck



Breaking the Bottleneck

- **The Flaw of Seq2Seq:** The network reads the whole sentence, compresses it into one vector, and then throws away the individual word memories.
- **The Proposed Fix:** What if we *don't* throw away the intermediate hidden states? What if we keep all of them: $h_1, h_2, \dots, h_3$?
- **The New Problem:** If the network has access to 100 different word states at once, how does it know which ones are actually important for the current word it is processing?
- To answer this, we are going to leave NLP entirely for a moment and look at signal processing.